

Distributed Systems

Lecture16: Multicast Communication

Mohammad Reza Shakournia

shakournia@ut.ac.ir

School of Electrical and Computer Engineering,
University of Tehran,
Tehran, Iran

Application-level Multicasting

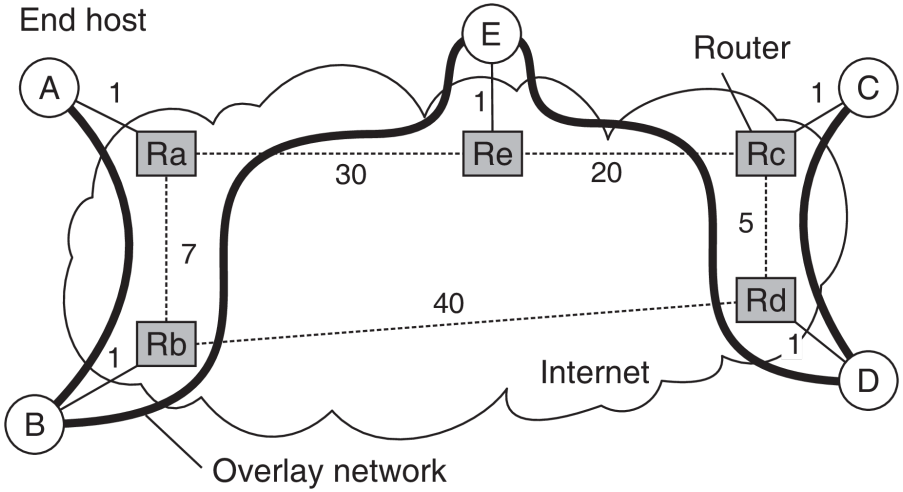
- Organize nodes of a distributed system into an overlay network and use that network to disseminate data:
  - Oftentimes a tree, leading to unique paths.
  - Alternatively, also mesh networks, requiring a form of routing.

Application-level Multicasting in Chord

- Basic approach:
  1. Initiator generates a multicast identifier mid.
  2. Lookup succ(mid), the node responsible for mid.
  3. Request is routed to succ(mid), which will become the root.
  4. If P wants to join, it sends a join request to the root.
  5. When request arrives at Q:
     - Q has not seen a join request before ⇒ it becomes forwarder; P becomes child of Q. Join request continues to be forwarded.
     - Q knows about tree ⇒ P becomes child of Q. No need to forward join request anymore.

ALM: Some Costs

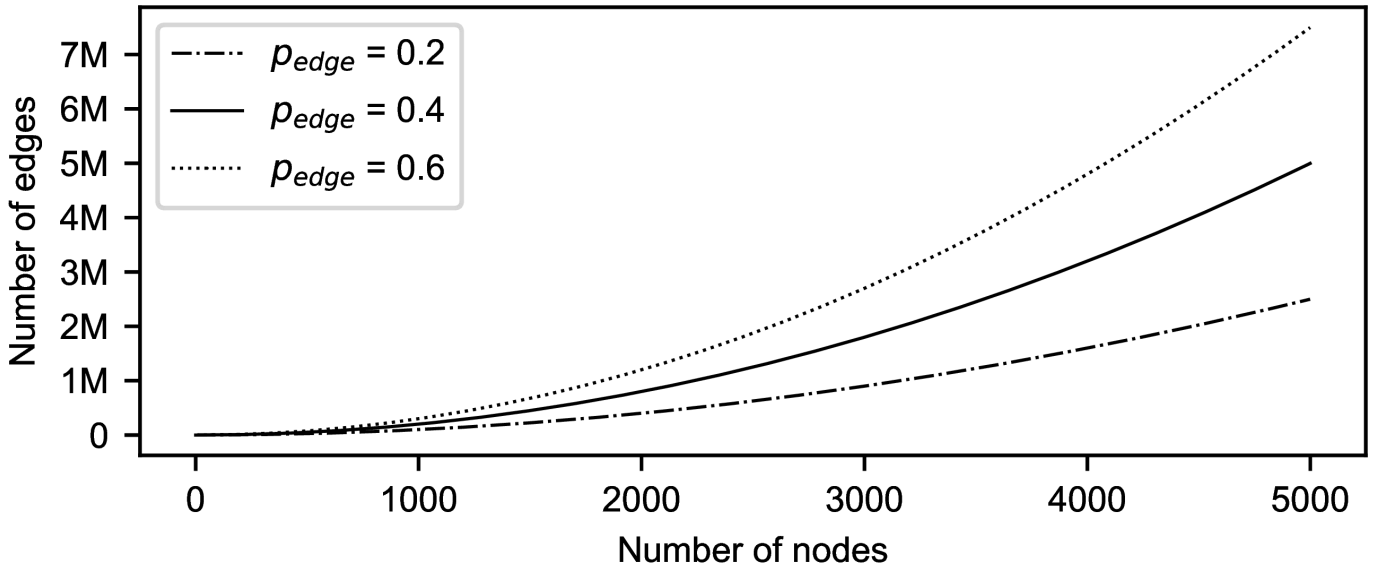
- Different metrics:



- **Link stress:** How often does an ALM message cross the same physical link?
  - Example: message from A to D needs to cross ⟨Ra,Rb⟩ twice.
- **Stretch:** Ratio in delay between ALM-level path and network-level path.
  - Example: messages B to C follow path of length 73 at ALM, but 47 at network level ⇒ stretch = 73/47.

Flooding

- P simply sends a message m to each of its neighbors. Each neighbor will forward that message, except to P, and only if it had not seen m before.



- Let Q forward a message with a certain probability p_{flood} , possibly even dependent on its own number of neighbors (i.e., node degree) or the degree of its neighbors.

Epidemic Protocols

- Assume there are no write–write conflicts:
  - Update operations are performed at a single server
  - A replica passes updated state to only a few neighbors
  - Update propagation is lazy, i.e., not immediate
  - Eventually, each update should reach every replica
- Two Forms of Epidemics:
  - Anti-entropy: Each replica regularly chooses another replica at random, and exchanges state differences, leading to identical states at both afterwards.
  - Rumor spreading: A replica which has just been updated (i.e., has been contaminated), tells several other replicas about its update (contaminating them as well).

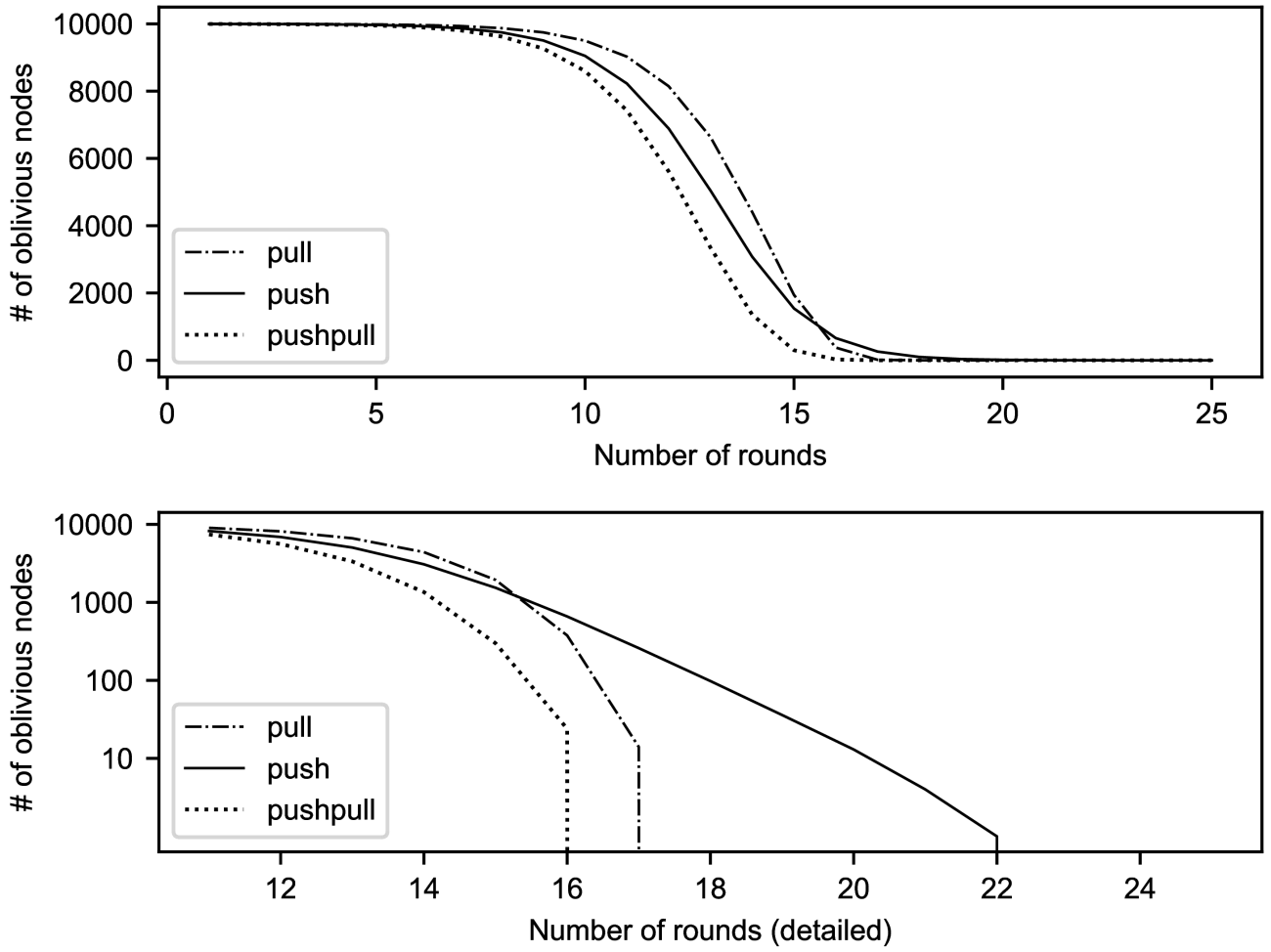
Anti-Entropy

- Principle operations:
  - A node P selects another node Q from the system at random.
  - Pull: P only pulls in new updates from Q
  - Push: P only pushes its own updates to Q
  - Push-pull: P and Q send updates to each other
- For push-pull it takes O(log(N)) rounds to disseminate updates to all N nodes (round = when every node has taken the initiative to start an exchange).

Anti-Entropy: Analysis

- Consider a single source, propagating its update. Let p_i be the probability that a node has not received the update after the ith round.
- Analysis: Staying Ignorant
- With pull, $p_{i+1} = (p_i)^2$: the node was not updated during the ith round and should contact another ignorant node during the next round.
- With push, $p_{i+1} = p_i(1 - \frac{1}{N-1})^{(N-1)(1-p_i)} \approx p_i e^{-1}$ (for small p_i and large N): the node was ignorant during the ith round and no updated node chooses to contact it during the next round.
- With push-pull: $(p_i)^2 . (p_i e^{-1})$

Anti-Entropy Performance



Rumor Spreading

- **Basic Model:**
  - A server S having an update to report, contacts other servers. If a server is contacted to which the update has already propagated, S stops contacting other servers with probability p_{stop}.
- **Observation:**
  - If s is the fraction of ignorant servers (i.e., which are unaware of the update), it can be shown that with many servers

$$s = e^{-\left(\frac{1}{p_{stop}+1}\right)(1-s)}$$

Formal Analysis

- **Notations:**
  - Let s denote fraction of nodes that have not yet been updated (i.e., susceptible; i the fraction of updated (infected) and active nodes; and r the fraction of updated nodes that gave up (removed).
- From Theory of Epidemics:

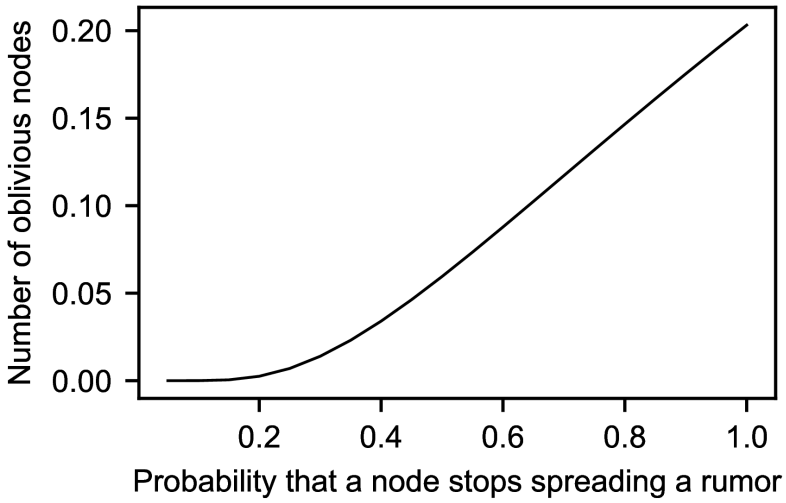
$$\begin{aligned} (1) \quad & ds/dt &=& \quad -s \cdot i \\ (2) \quad & di/dt &=& \quad s \cdot i - p_{stop} \cdot (1-s) \cdot i \\ \Rightarrow \quad & di/ds &=& \quad -(1+p_{stop}) + \frac{p_{stop}}{s} \\ \Rightarrow \quad & i(s) &=& \quad -(1+p_{stop}) \cdot s + p_{stop} \cdot \ln(s) + C \end{aligned}$$

- $i(1) = 0 \Rightarrow C = 1 + p_{stop} \Rightarrow$
- $i(s) = (1 + p_{stop})(1 - s) + p_{stop}.\ln(s).$
- We are looking for the case $i(1) = 0$, which leads to

$$s = e^{-\left(\frac{1}{p_{stop}+1}\right)(1-s)}$$

Rumor Spreading

- Rumor spreading:



Consider 10,000 nodes		
$1/p_{stop}$	s	N_s
1	0.203188	2032
2	0.059520	595
3	0.019827	198
4	0.006977	70
5	0.002516	25
6	0.000918	9
7	0.000336	3

- If we really have to ensure that all servers are eventually updated, rumor spreading alone is not enough.

Deleting Values

- **Fundamental problem:**
  - We cannot remove an old value from a server and expect the removal to propagate. Instead, mere removal will be undone in due time using epidemic algorithms.
- **Solution:**
  - Removal has to be registered as a special update by inserting a death certificate.

Deleting Values

- When to remove a death certificate (it is not allowed to stay for ever):
  - Run a global algorithm to detect whether the removal is known everywhere, and then collect the death certificates (looks like garbage collection)
  - Assume death certificates propagate in finite time, and associate a maximum lifetime for a certificate (can be done at risk of not reaching all servers)
- **Note:** It is necessary that a removal actually reaches all servers.

Chapter Wrap-Up

In the next chapter, we will discuss how processes can synchronize and coordinate their actions. We also take a look at below topics:

- Clock Synchronization,
- Logical Clocks,
- Mutual Exclusion,
- Election Algorithms,
- Gossip-based Coordination,
- Distributed Event Matching,
- Location Systems.